DEVOPS FINAL CAPSTONE

ClinicFlow

A clinic appointment system taken from code all the way to a monitored, autoscaled Kubernetes deployment

FastAPI | React | PostgreSQL | Docker | GitHub Actions | Trivy | Terraform | AWS EKS | Kubernetes | Helm | Argo CD | Prometheus | Grafana



Name: Harshita Hirawat

Roll no: 24BCS10044

github.com/Harshita-008/devops-capstone-clinic

THE APPLICATION

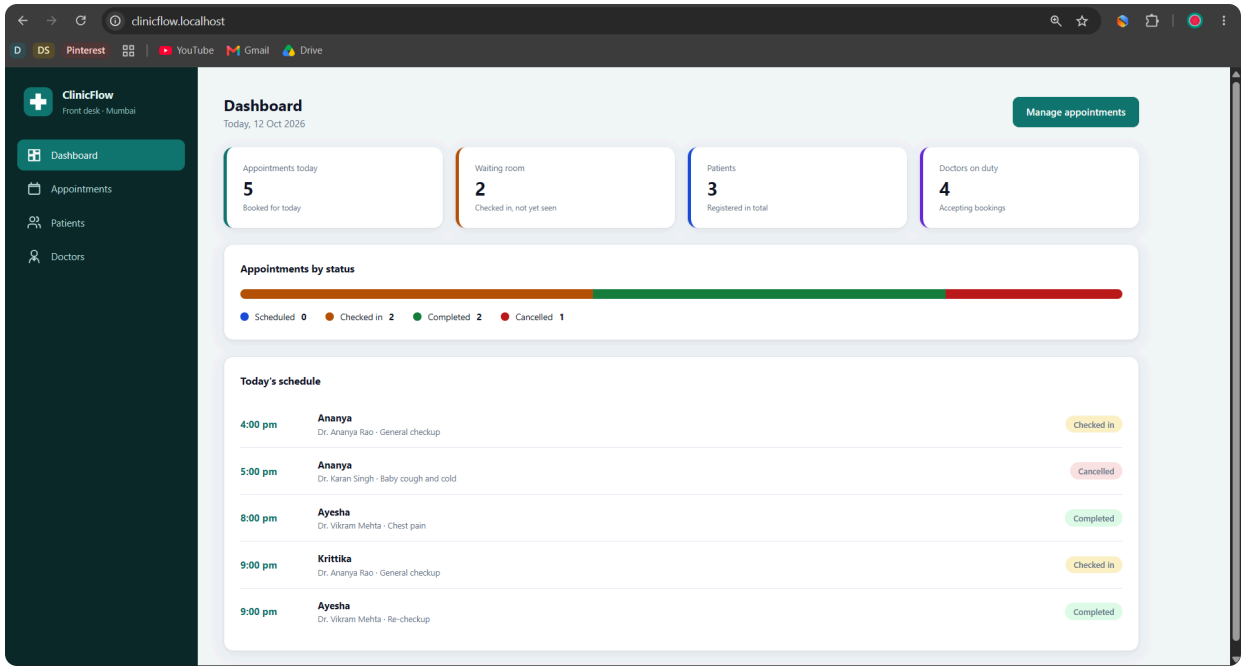
What ClinicFlow does

A front desk system for a small clinic. Staff register patients, manage doctors and book appointments, then move each visit through the day.

Scheduled Checked in Completed Cancelled

Clinic rules built into the API

- No double booking of a doctor within 30 minutes
- No bookings in the past or with an unavailable doctor
- Patients with history cannot be deleted, it is a medical record



Dashboard: KPIs, status breakdown and today's schedule

BIG PICTURE

How everything fits together

DELIVERY

git push
code on GitHub

↓

GitHub Actions
test, build, scan, push

↓

GHCR
images tagged with commit SHA

↓

Set tag
new SHA in values.yaml, push

↓

Argo CD
syncs the cluster from Git

KUBERNETES CLUSTER (KIND)

Traefik Ingress
clinicflow.localhost

→ **Frontend**
React + nginx · 2 pods · route /

→ **Backend**
FastAPI · 2 to 5 pods (HPA) · route /api

→ **PostgreSQL**
StatefulSet + volume

Prometheus
scrapes /metrics

→ **Grafana**
ClinicFlow dashboard

Terraform → AWS Mumbai: **VPC** with 2 public and 2 private subnets, and an **EKS cluster** with a node group. Created, verified and destroyed.

TOOLS

Tech stack

Application

React + Vite frontend
FastAPI backend (Python 3.13)
PostgreSQL 17 with **SQLAlchemy** and **Alembic**

Quality

pytest with coverage
ruff for Python linting
ESLint for the frontend

Containers

Docker multi-stage builds
Docker Compose for local runs
non-root images

CI/CD and security

GitHub Actions pipeline
GHCR image registry
Trivy vulnerability scan

Infrastructure

Terraform
AWS VPC and **EKS**
region ap-south-1 (Mumbai)

Kubernetes

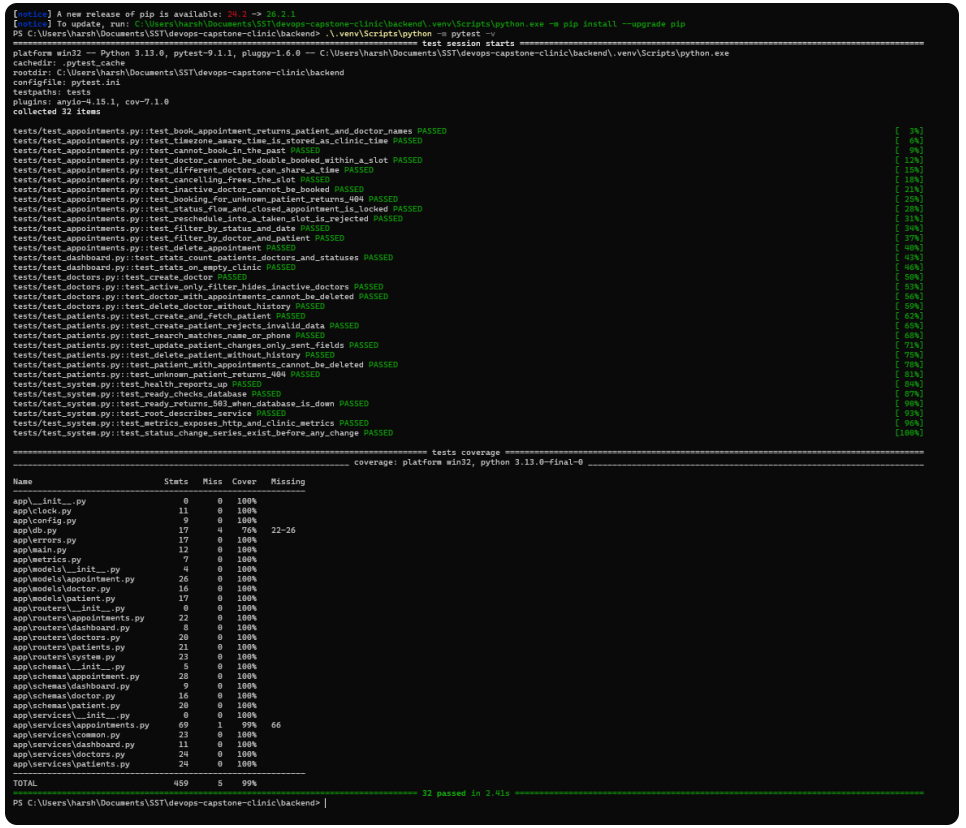
kind, **Helm**, **Traefik** Ingress
HPA with metrics-server
Argo CD, **Prometheus**, **Grafana**

BACKEND AND TESTING

A clean API with tests as the first gate

- **19 API endpoints**: create, read, update and delete for patients, doctors and appointments, dashboard stats, and `/health` and `/ready` checks
- Code split into layers: **routers** (HTTP), **services** (rules), **models** (tables), **schemas** (data shapes)
- **Alembic** migrations create the tables and seed 4 doctors
- **32 tests, 99% coverage**, on an in-memory SQLite database, so the real database is never touched

If a single test fails, the pipeline stops and **no image is built**.



pytest -v: 32 passed, 99% coverage

CONTAINERS

Docker: small, safe images

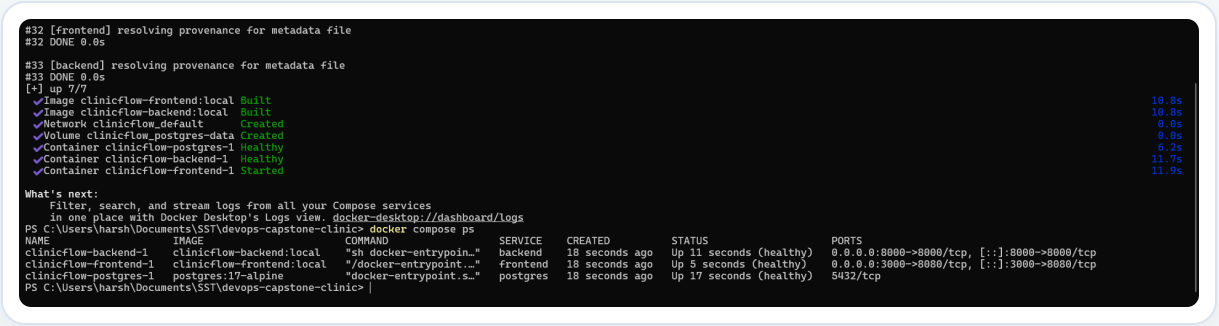
Backend

Two stages: install packages, then copy only what runs. Runs as user `10001`. Pip removed.

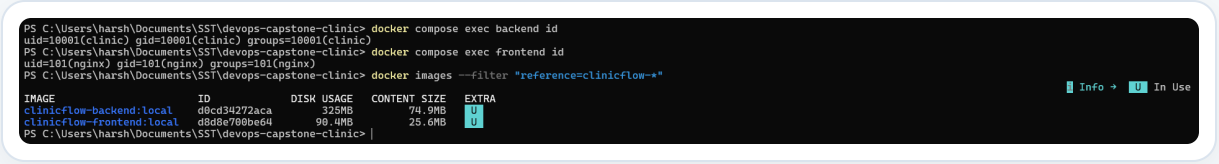
Frontend

Node builds the React app, then only **nginx + static files** ship. Runs as user `101`.

- `docker compose up --build` starts frontend, backend and PostgreSQL
- Healthchecks make each service wait for the one it depends on
- Frontend image content is only **25.6 MB** because Node is left out



All three services healthy



Non-root users and image sizes

CI/CD

GitHub Actions pipeline

1 Test
ruff + pytest
ESLint + build

→

2 Build
backend and frontend images

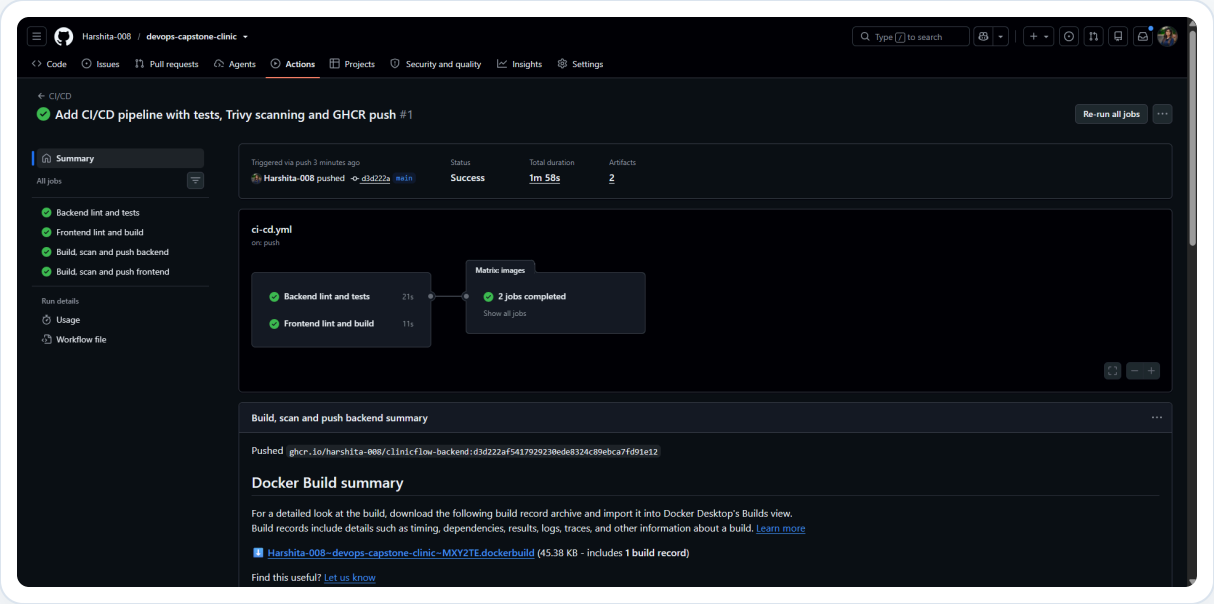
→

3 Scan
Trivy fails on HIGH / CRITICAL

→

4 Push
GHCR, tag = commit SHA

- Runs on every code push and pull request to `main`
- Images are tagged with the **commit SHA**, never `latest`, so every pod traces back to one commit
- Pull requests build and scan, but do not push
- Every action is pinned to an exact commit for supply chain safety



A green pipeline run: tests, then both images built, scanned and pushed

DEVSECOPS

Trivy found real vulnerabilities

First scan: 4 HIGH

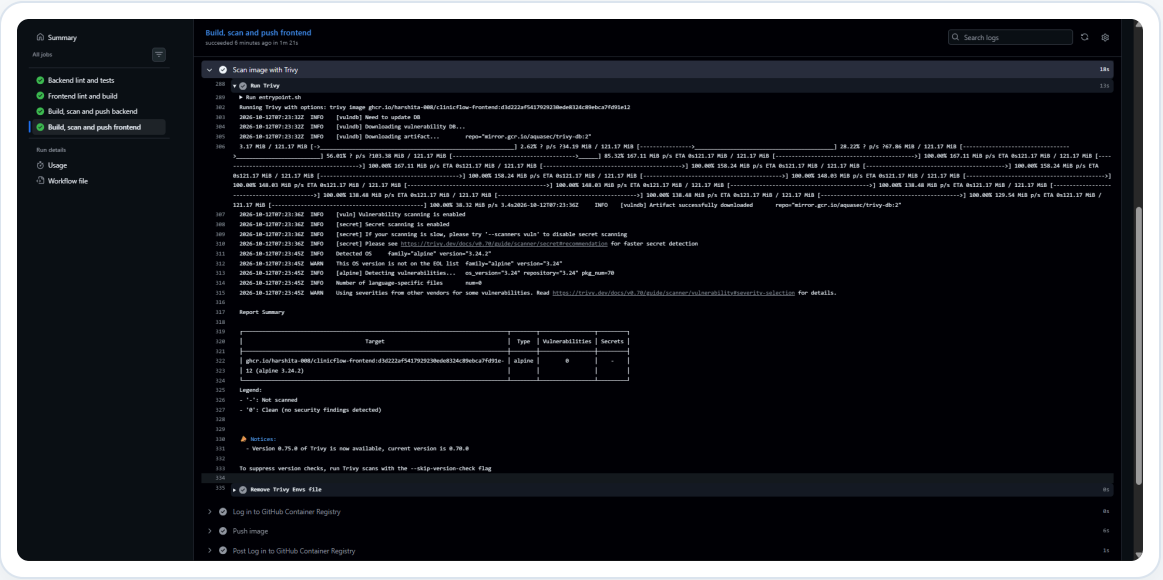
Found in `urllib3`, `msgpack` and `setuptools`. For example **CVE-2026-97687** in urllib3 allows traffic interception through an HTTPS proxy.

Root cause

They were not my dependencies. They came **bundled inside pip**, which the app never uses at runtime.

Fix: 0 vulnerabilities

Removed pip from the final image. Both images now scan clean, and the backend got 17 MB smaller.



Trivy step in the pipeline: 0 HIGH or CRITICAL

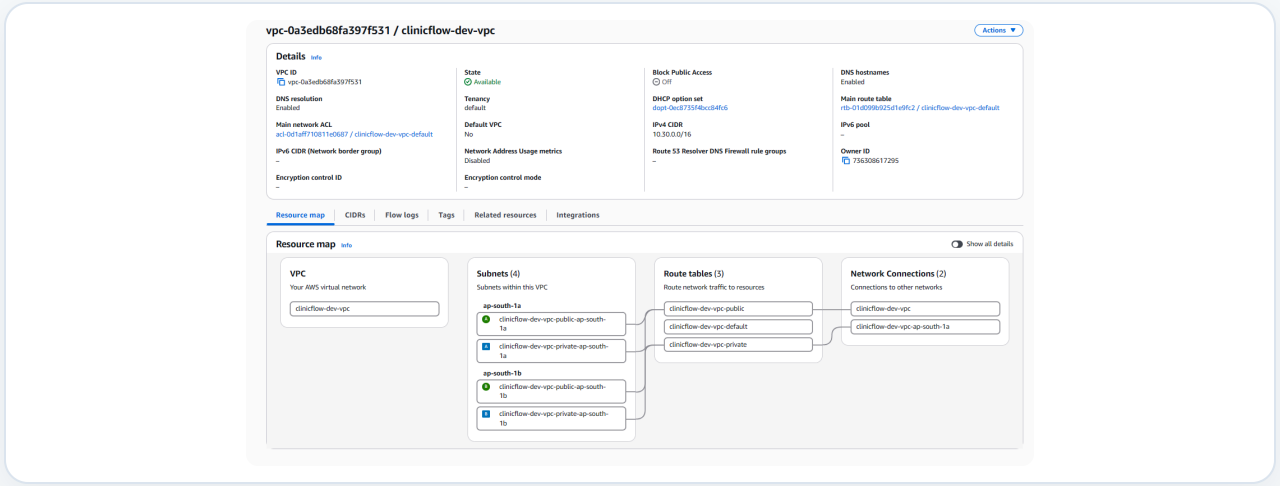
INFRASTRUCTURE AS CODE

Terraform on AWS

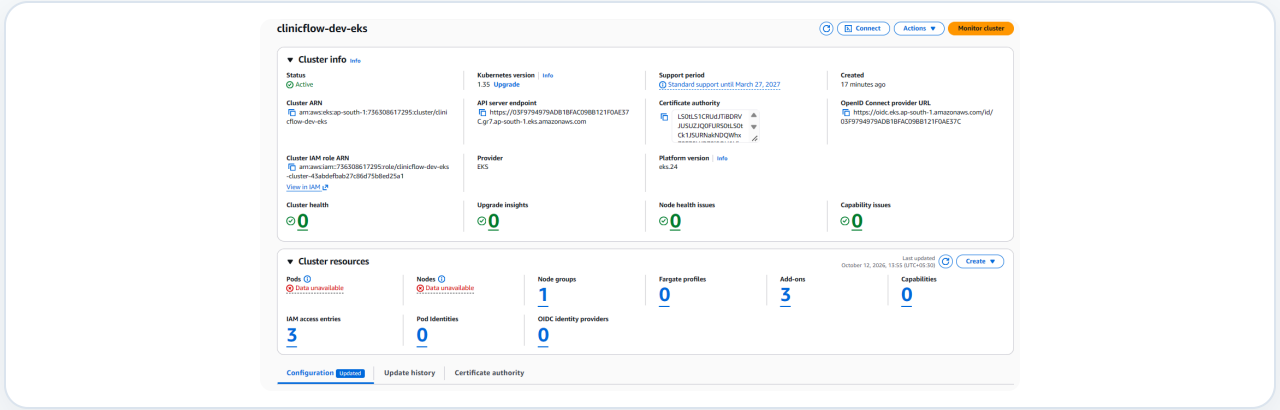
- **VPC** with 2 public and 2 private subnets in 2 availability zones, an internet gateway and one NAT gateway
- **EKS 1.35** cluster with a managed node group
- Credentials come from the AWS CLI, never from the code

plan 50 resources → **apply** ~15 min → **verify** console + kubectl → **destroy** 50 removed

Cost aware: one NAT gateway, one small node, a standard support version, all destroyed in the same session.



VPC resource map: 4 subnets, route tables, gateways



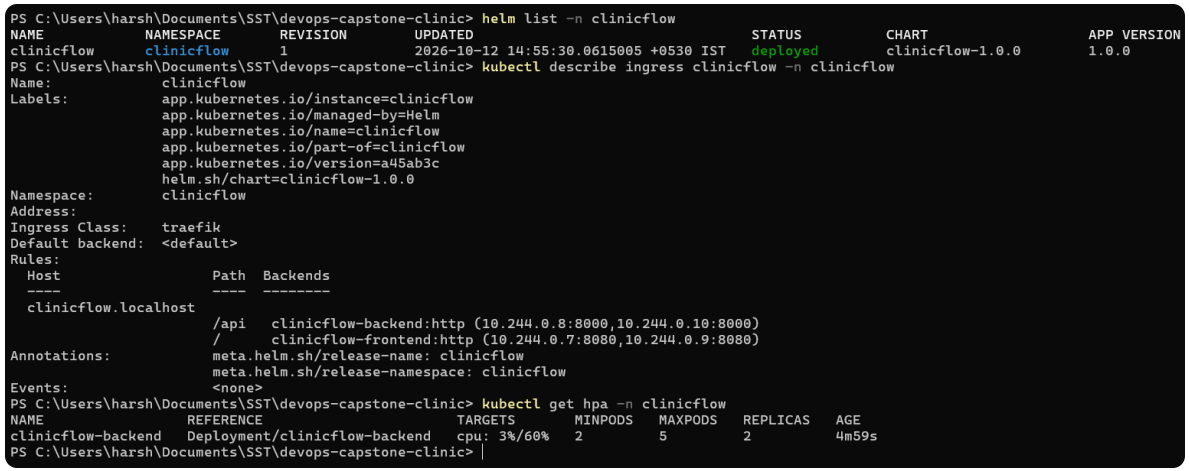
EKS cluster Active

KUBERNETES

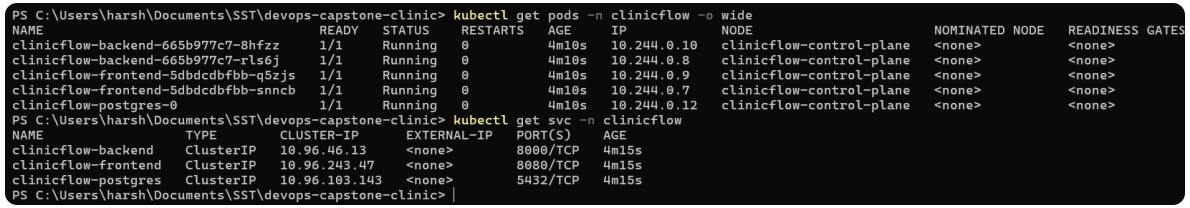
Kubernetes with a Helm chart

Deployments	backend and frontend, 2 replicas each
Services	ClusterIP for backend, frontend, PostgreSQL
StatefulSet	PostgreSQL with a persistent volume
Ingress	`/api` to backend, `/` to frontend
Probes	`/health` for liveness, `/ready` for readiness

Migrations run in an **init container** with a database lock, so two pods starting together never migrate at the same time.



helm list, Ingress rules and HPA



All pods Running

AUTOSCALING

The backend scales by itself under load

2 → 5
backend pods

~60s
to scale out

- The HPA targets **60% CPU**, with 2 to 5 pods
- A load test Job sends requests for 3 minutes
- CPU went above 400% of the target, so the HPA added pods
- After the load stopped, it scaled back down to 2

```
PS C:\Users\harsh\Documents\SST\devops-capstone-clinic> kubectl get hpa -n clinicflow -w
NAME                 REFERENCE                          TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
clinicflow-backend   Deployment/clinicflow-backend   cpu: 2%/60%   2         5         2          21m
clinicflow-backend   Deployment/clinicflow-backend   cpu: 32%/60%   2         5         2          22m
clinicflow-backend   Deployment/clinicflow-backend   cpu: 372%/60%   2         5         2          22m
clinicflow-backend   Deployment/clinicflow-backend   cpu: 438%/60%   2         5         4          22m
clinicflow-backend   Deployment/clinicflow-backend   cpu: 389%/60%   2         5         5          22m
clinicflow-backend   Deployment/clinicflow-backend   cpu: 234%/60%   2         5         5          23m
PS C:\Users\harsh\Documents\SST\devops-capstone-clinic>
```

kubectl get hpa -w: replicas 2, 4, 5

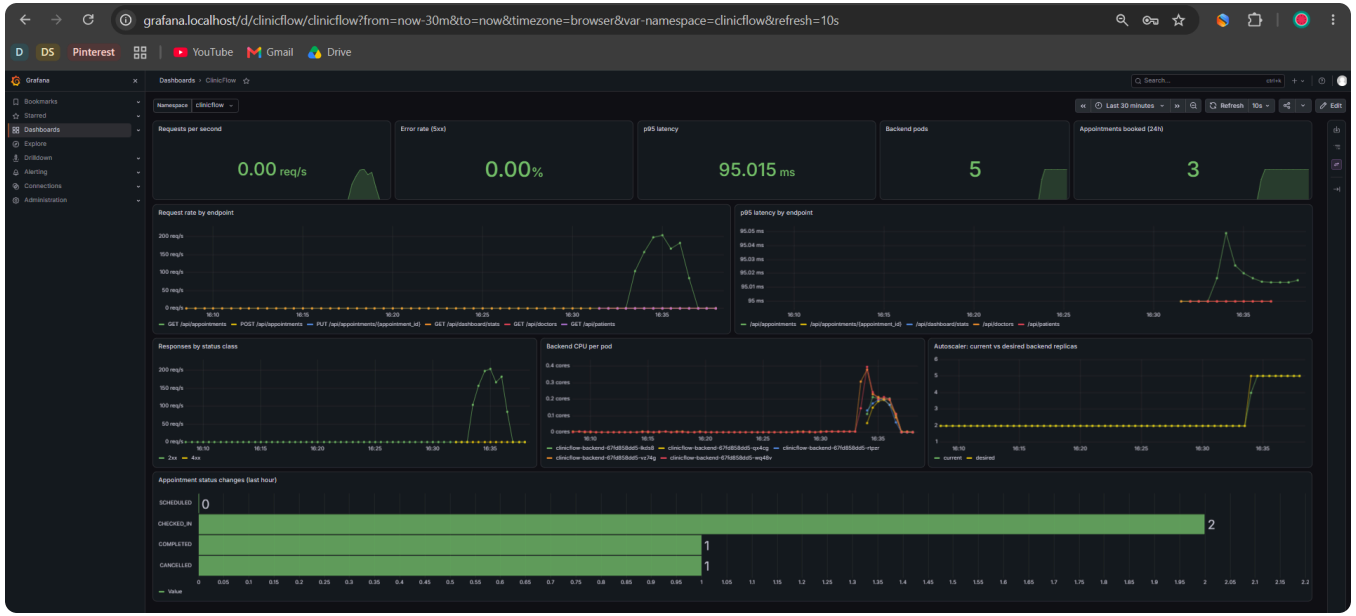
```
PS C:\Users\harsh\Documents\SST\devops-capstone-clinic> kubectl get pods -n clinicflow
NAME                                 READY   STATUS    RESTARTS   AGE
clinicflow-backend-665b977c7-8hfzz   1/1     Running   0          24m
clinicflow-backend-665b977c7-l59pp   1/1     Running   0          114s
clinicflow-backend-665b977c7-nrx8z   1/1     Running   0          99s
clinicflow-backend-665b977c7-rls6j   1/1     Running   0          24m
clinicflow-backend-665b977c7-x4cmm   1/1     Running   0          114s
clinicflow-frontend-5dbdcdbfbb-q5zjs   1/1     Running   0          24m
clinicflow-frontend-5dbdcdbfbb-snncb   1/1     Running   0          24m
clinicflow-load-test-2jg96           1/1     Running   0          2m40s
clinicflow-load-test-6zn5h           1/1     Running   0          2m40s
clinicflow-load-test-8qcx2           1/1     Running   0          2m40s
clinicflow-load-test-nrpks           1/1     Running   0          2m40s
clinicflow-postgres-0                1/1     Running   0          24m
PS C:\Users\harsh\Documents\SST\devops-capstone-clinic> kubectl top pods -n clinicflow
NAME                                 CPU(cores)   MEMORY(bytes)
clinicflow-backend-665b977c7-8hfzz   193m         67Mi
clinicflow-backend-665b977c7-l59pp   195m         67Mi
clinicflow-backend-665b977c7-nrx8z   189m         66Mi
clinicflow-backend-665b977c7-rls6j   203m         67Mi
clinicflow-backend-665b977c7-x4cmm   193m         66Mi
clinicflow-frontend-5dbdcdbfbb-q5zjs   1m           10Mi
clinicflow-frontend-5dbdcdbfbb-snncb   1m           10Mi
clinicflow-load-test-2jg96           199m         0Mi
clinicflow-load-test-6zn5h           199m         1Mi
clinicflow-load-test-8qcx2           200m         0Mi
clinicflow-load-test-nrpks           199m         1Mi
clinicflow-postgres-0                241m         72Mi
PS C:\Users\harsh\Documents\SST\devops-capstone-clinic>
```

5 backend pods sharing the load

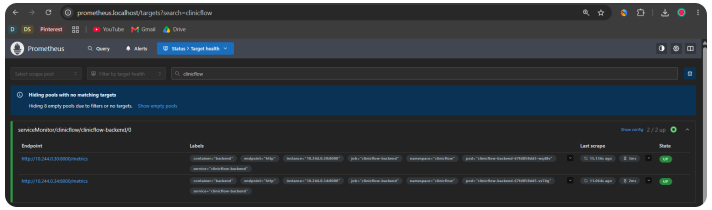
OBSERVABILITY

Prometheus and Grafana

- The backend exposes `/metrics`
- A **ServiceMonitor** tells Prometheus to scrape it every 15 seconds
- The Grafana dashboard is **stored in the Helm chart** and loads automatically
- Panels: traffic, errors, p95 latency, CPU, pods and the HPA
- Plus clinic metrics: bookings and status changes



Grafana during the load test



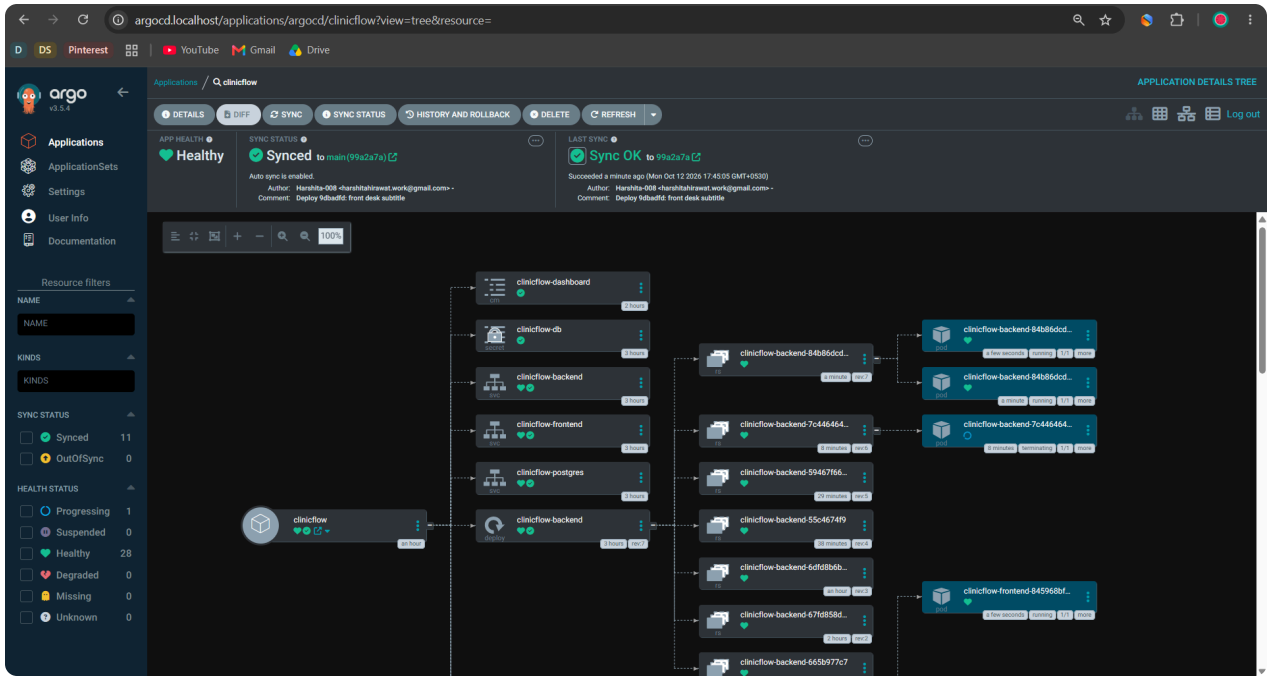
Prometheus target: 2/2 UP

CONTINUOUS DEPLOYMENT

From commit to live with Argo CD

1. **Push code** — change the UI text
2. **Pipeline** — test, build, scan, push
3. **Set tag** — new SHA in values.yaml
4. **Argo CD** — syncs from Git
5. **Live** — pods replaced one by one

- GitHub cannot reach my laptop cluster, so the cluster **pulls from Git** instead
- **Auto sync**, **prune** and **self heal** keep the cluster equal to Git
- Every deployment is in the history and can be rolled back



Synced and Healthy, deployed from my commit

FAILURE SIMULATION

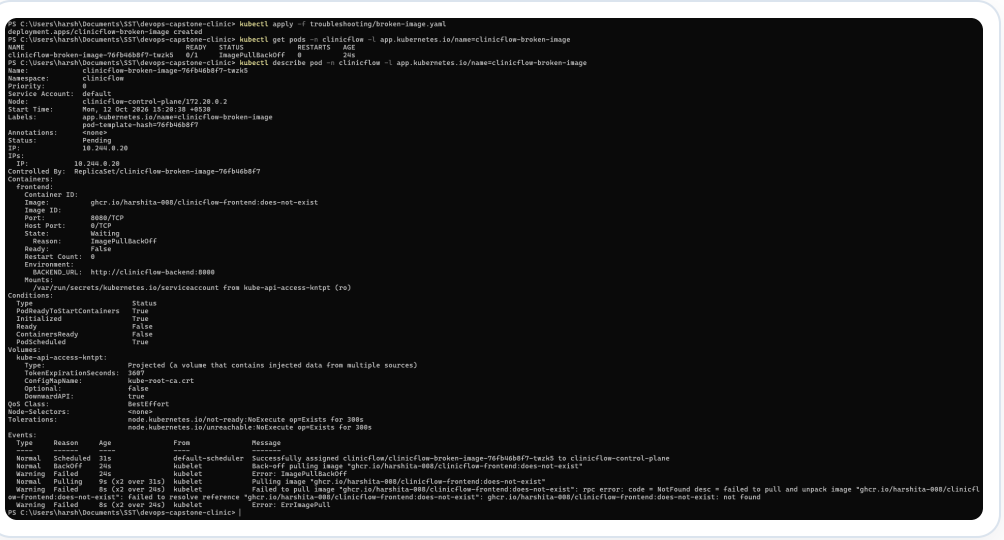
Troubleshooting lab

Scenario 1: wrong image tag

Symptom: pod stuck in `ImagePullBackOff`

Found with: pod events show the tag was not found

Fix: `kubectl set image` to a real tag



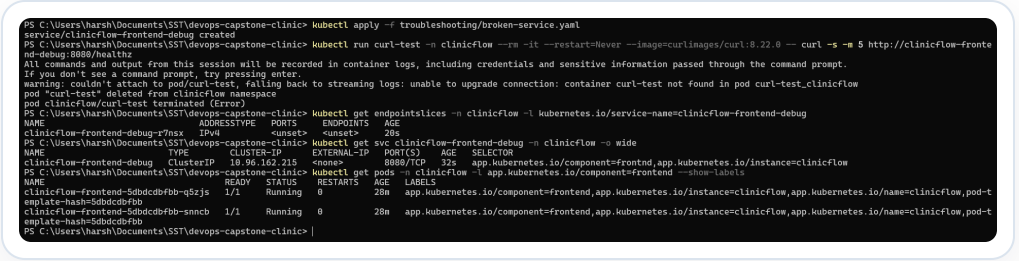
kubectl describe: the pull error in the events

Scenario 2: Service with no endpoints

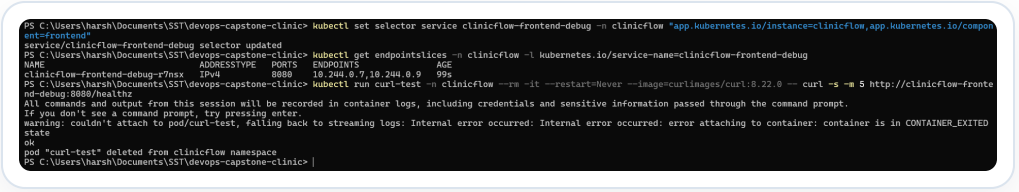
Symptom: requests to the Service time out

Found with: empty EndpointSlice, selector typo `frontnd`

Fix: `kubectl set selector`, endpoints appear



Diagnosis: no endpoints, selector says frontnd



After the fix: two endpoints and the request works

LEARNINGS

Problems I hit and how I solved them

Vulnerabilities inside pip

Trivy flagged 4 HIGH CVEs. Removing pip from the runtime image fixed all of them.

Two pods migrating at once

Replicas could create the same tables together. A PostgreSQL lock makes them take turns.

A panel always showing 0

Prometheus missed the first status change. Starting every counter at 0 fixed the Grafana panel.

ingress-nginx archived

The course controller no longer gets security fixes, so I used Traefik with the same Ingress resource.

Health check failing in nginx

`localhost` pointed to IPv6 inside the container. Checking `127.0.0.1` fixed it.

Keeping AWS cost near zero

Small node, one NAT gateway, a standard support version, and destroy right after verifying.

SUMMARY

What the project covers

19
API endpoints

32
tests, 99% coverage

0
HIGH / CRITICAL CVEs

2→5
pods with autoscaling

AREA	WHAT I BUILT
App and tests	React + FastAPI + PostgreSQL, Alembic migrations, pytest
Delivery	Docker, GitHub Actions, Trivy, GHCR with SHA tags
Infrastructure	Terraform VPC + EKS on AWS, created and destroyed
Operations	Kubernetes, Helm, Ingress, HPA, Argo CD, Prometheus and Grafana

Every step is backed by a screenshot and the exact commands in the repository README.

Thank you

github.com/Harshita-008/devops-capstone-clinic

